In [2]:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load and prepare data
df =
pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
df['Age'] = df['Age'].fillna(df['Age'].median())

features = ['Age', 'Pclass', 'Sex', 'Fare']
target = 'Survived'
X = df[features]
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Train the model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Calculate the confusion matrix
cm = confusion_matrix(y_test, y_pred)

# 2. Create a heatmap to visualize it
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Died',
'Survived'], yticklabels=['Died', 'Survived'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

from sklearn.metrics import classification_report

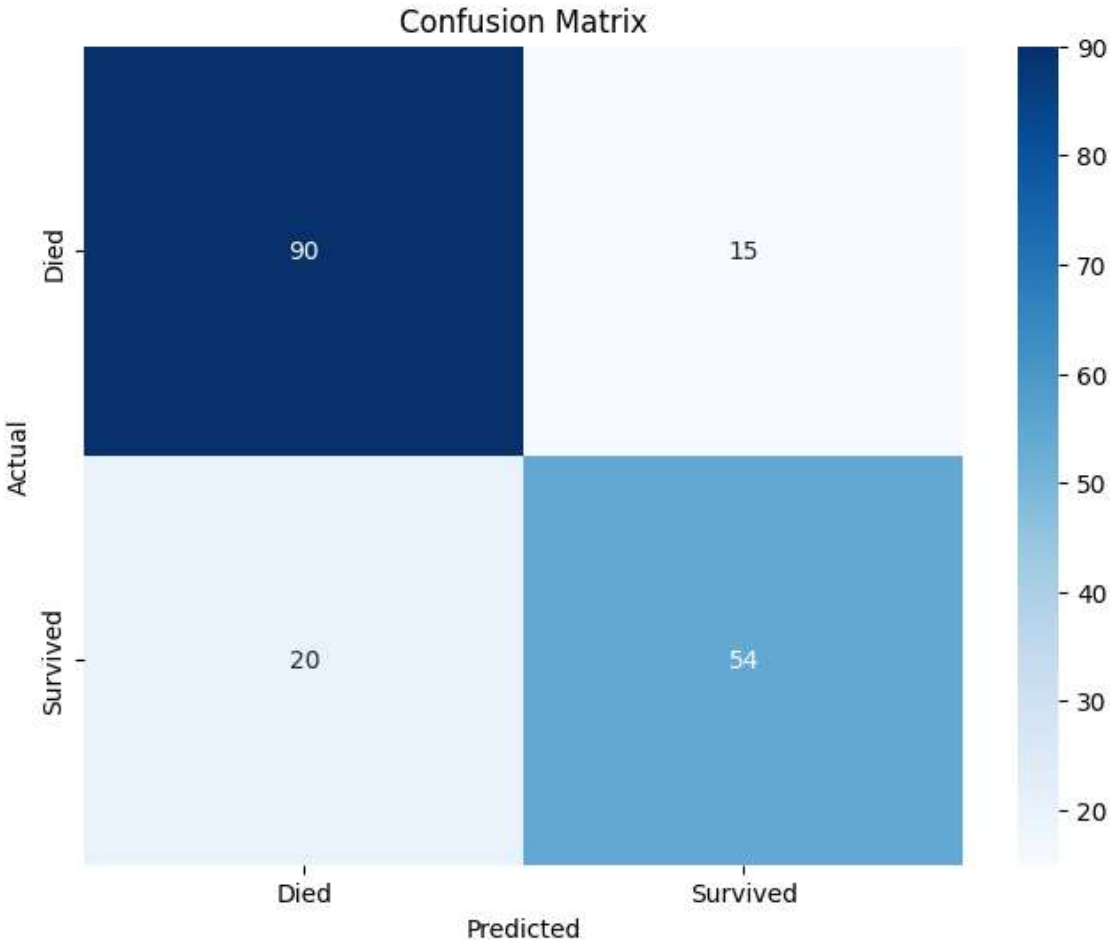
# 1. Generate and print the classification report
report = classification_report(y_test, y_pred)
print(report)

from sklearn.model_selection import cross_val_score

# 1. Perform 5-fold cross-validation
cv_scores = cross_val_score(model, X, y, cv=5)

# 2. Print the scores for each fold and the average score
```

```
print(f"Scores for each fold: {cv_scores}")
print(f"Average CV Score: {cv_scores.mean():.2%}")
print(f"Standard Deviation of CV Scores: {cv_scores.std():.4f}")
```



	precision	recall	f1-score	support
0	0.82	0.86	0.84	105
1	0.78	0.73	0.76	74
accuracy			0.80	179
macro avg	0.80	0.79	0.80	179
weighted avg	0.80	0.80	0.80	179

```
Scores for each fold: [0.7877095  0.78089888 0.78651685 0.7752809
0.80337079]
Average CV Score: 78.68%
Standard Deviation of CV Scores: 0.0094
```

Knowledge Check

1. Describe a real-world scenario where you would care more about a model's Precision than its Recall.

Email Spam Filter: In this scenario, a false positive means an important email from a boss, school, or friend gets mistakenly sent to the spam folder. You care more about precision here because you want to ensure that when the model labels an email as spam, it is almost certainly spam, avoiding the risk of missing critical messages.

2. Describe a real-world scenario where you would care more about a model's Recall than its Precision.

Medical Diagnosis (e.g., detecting a serious disease like cancer): Here, a false negative means a sick patient is incorrectly told they are healthy. You care much more about recall because you want to catch every single positive case of the disease, even if it means dealing with a few false alarms (false positives) that get sorted out in follow-up tests.

3. Why is a cross-validation score generally more trustworthy than a score from a single train-test split?

A single train-test split depends entirely on how the data happens to be divided, which can sometimes lead to a lucky or unlucky split that doesn't reflect the model's true capability. Cross-validation splits the data into multiple different folds, trains and tests the model across all of them, and averages the results. This reduces bias and variance, giving a much more reliable and robust estimate of how the model will perform on completely unseen data.